

RAG • CHUNKING • EMBEDDINGS • HYBRID RETRIEVAL • EVALUATION • KNOWLEDGE GRAPHS • AGENTIC

Cracking RAG and GraphRAG System Design Interviews

From Basic RAG Fundamentals to Agentic GraphRAG
with LangChain, LangGraph, Neo4j, pgvector, and Ollama

AI ENGINEERING INSIDER • aiengineeringinsider.com

10 CHAPTERS • 70 SENIOR INTERVIEW QUESTIONS • RUNNABLE LABS



Cracking RAG and GraphRAG System Design Interviews

First edition, 2026.

Published by AI Engineering Insider.

Web: aiengineeringinsider.com

Newsletter: aiengineeringinsider.substack.com

Companion code repository: github.com/lamhotsiagian/rag-graphrag-lab

All code in this book is provided for educational use. Configuration values, credentials, and connection strings shown in examples are development defaults and must be replaced before any production deployment.

Product and company names mentioned in this book are the trademarks of their respective owners. Reference to them is made for identification and educational purposes only and does not imply endorsement.

The performance figures, latency ranges, and cost estimates in this book are indicative values drawn from typical deployments. Measure them on your own workload before relying on them for a design decision.

Preface

Subscribe → aiengineeringinsider.substack.com/subscribe

Most material on Retrieval Augmented Generation stops at the tutorial. It shows you how to embed a document, search a vector store, and paste the results into a prompt. That knowledge gets you a working demo in an afternoon, and it gets you rejected in a system design interview by lunchtime the next day.

This book covers the other ninety percent. It treats RAG as a distributed system with an offline plane and an online plane, with latency budgets that decompose per stage, with failure modes that have names, and with trade-offs you must be able to defend under questioning. Every chapter builds a component, critiques the obvious implementation of that component, and then shows the version that survives production.

What Makes This Book Different

It is built around a working codebase. Every chapter maps to a runnable lab in the companion repository at github.com/lamhotsiagian/rag-graphrag-lab. The code runs entirely on your own machine using Ollama, PostgreSQL with pgvector, and Neo4j, so nothing in this book requires a hosted API key or a cloud account.

It critiques its own code. Several chapters deliberately examine a naive implementation, name the specific defect, and then present the corrected version. Those defects are not invented. They are the ones that appear in real code reviews, and recognizing them is a large part of what senior interviews test.

It is interview shaped. Each chapter closes with seven questions drawn from the categories interviewers actually use: open system design, production debugging, technical depth, real production case studies, trade-off defence, and operational readiness. The answers are written at the length and depth a strong candidate would actually deliver.

It refuses shallow definitions. Where a concept has a mechanism, the mechanism is explained. Where a recommendation has a cost, the cost is stated. Where a number matters, the number is given along with the assumption behind it.

Who This Book Is For

You will get the most from this book if you write code and expect to defend architecture. That includes AI engineers building retrieval systems, machine learning engineers moving into applied LLM work, software engineers taking ownership of an AI feature, agentic AI engineers designing tool-using workflows, and architects reviewing designs they did not write.

You need working Python, comfort with SQL, and familiarity with basic distributed systems vocabulary. You do not need prior experience with vector databases, graph databases, or Lang-Graph, all of which this book builds from first principles.

How to Use This Book

Read Part I in order, because every later chapter depends on the four stage model and the retriever contract established there. After that, the parts are more independent: Part II covers retrieval quality and measurement, Part III covers knowledge graphs, Part IV covers agentic orchestration, and Part V covers production and interview practice.

Run the labs. Reading about a similarity score teaches you less in an hour than watching one drop when you paraphrase a query teaches you in a minute. Each chapter includes a testing walkthrough written specifically to make a concept visible in the user interface.

Answer the interview questions before reading the answers. Write your response, then compare. The gap between what you wrote and what is written is the specific material worth rereading.

Running the Labs

```
1 # 1. Clone the companion repository
2 git clone https://github.com/lamhotsiagian/rag-graphrag-lab.git
3 cd rag-graphrag-lab
4
5 # 2. Install Python dependencies
6 python -m venv .venv && source .venv/bin/activate
7 pip install -r requirements.txt
8
9 # 3. Start PostgreSQL with pgvector and Neo4j
10 docker compose up -d
11
12 # 4. Pull the local models
13 ollama pull qwen2.5v1:3b
14 ollama pull nomic-embed-text
15
16 # 5. Copy the environment template and adjust if needed
17 cp .env.example .env
18
19 # 6. Run any chapter lab
20 streamlit run chapter_01/app.py
21
22 # 7. Run the test suite
23 ./scripts/run_tests.sh
```

AI Engineering Insider

aiengineeringinsider.com • aiengineeringinsider.substack.com

Chapter Map

Subscribe → aiengineeringinsider.substack.com/subscribe

Use this table to navigate directly to the material you need. Every chapter is self-contained enough to read alone once you have finished Chapter 1.

Ch	Title	Main focus	Lab
1	RAG Foundations and System Design Fundamentals	Hallucination mechanics, the four stages, offline against on-line planes, requirements and the accuracy, latency, scalability, cost trade-off	chapter_01 , in-memory RAG v1
2	Document Ingestion, Parsing, and Chunking	Loaders per format, safe normalization, metadata extraction, four chunking strategies, the ingestion ceiling	chapter_02 , chunking comparison
3	Embeddings and Vector Database Architecture	Embedding geometry, dense against sparse, similarity measures, HNSW and IVFFlat, metadata filtering and pre-filter collapse	chapter_03 , semantic search
4	Building a Complete Basic RAG System	Retriever contract, deterministic context construction, grounded prompting, citation validation, conversation-aware retrieval	chapter_04 , full RAG application
5	Advanced Retrieval and Hybrid RAG	BM25, reciprocal rank fusion, cross encoder reranking, query rewriting, multi-query, HyDE, corrective and adaptive RAG	chapter_05 , retriever comparison
6	RAG Evaluation, Observability, and Debugging	Retrieval and generation metrics, claim-level faithfulness, golden datasets, label-free on-line monitoring, failure analysis	chapter_06 , evaluation dashboard
7	Knowledge Graph Fundamentals	Property graph modelling, Cypher traversal, constrained extraction with provenance, entity resolution, hub detection	chapter_07 , graph builder

Ch	Title	Main focus	Lab
8	GraphRAG and Hybrid Graph Retrieval	Local and global search, bounded multi-hop traversal, community detection, unified reranking fusion, four-architecture comparison	chapter_08 , GraphRAG comparison
9	Agentic GraphRAG and Multi-Hop Reasoning	LangGraph state and reducers, conditional routing, bounded cycles, checkpointing, verification, human-in-the-loop	chapter_09 , agentic workflow
10	Production RAG and GraphRAG System Design Interviews	Requirements framework, scalability, reliability, security, cost levers, six complete interview scenarios	chapter_10 , design practice

Contents

Preface	ii
Chapter Map	iv
List of Figures	xiv
List of Tables	xvi
Part I RAG Foundations	1
1 RAG Foundations and System Design Fundamentals	3
1.1 Why Language Models Hallucinate	3
1.2 Why the Context Window Is Not a Database	4
1.3 The Four Stages of a RAG System	5
1.4 Offline Indexing Compared With Online Serving	5
1.5 Naive RAG and Its Failure Surface	7
1.6 Framing RAG as a System Design Problem	7
1.7 Building RAG v1	8
1.8 Testing RAG v1 Through the User Interface	11
1.9 Chapter Summary	12
1.10 Interview Questions and Answers	12
Q1.1 Design a document question answering system for 50,000 internal documents	13
Q1.2 Answers are confident and wrong. How do you localize the fault?	13
Q1.3 Why not simply use a one million token context window?	14
Q1.4 The customer support bot regressed after a model upgrade	15
Q1.5 How do you choose K , and what breaks at the extremes?	15
Q1.6 Design the abstention policy and defend it against product pressure	16
Q1.7 Take this design from 50,000 documents to 10 million	17
2 Document Ingestion, Parsing, and Chunking	19
2.1 Ingestion Is Where Retrieval Quality Is Decided	19
2.2 Document Loaders by Format	19
2.3 Cleaning and Normalization Without Destroying Signal	21
2.4 Metadata Extraction	22
2.5 Chunking Strategies	22
2.6 Chunk Size and Overlap Trade-offs	24
2.7 Building the Chunking Comparison Lab	25
2.8 Testing Ingestion Through the User Interface	27
2.9 Chapter Summary	28
2.10 Interview Questions and Answers	28

Q2.1	Design the ingestion pipeline for a 5 million document corpus with mixed formats	29
Q2.2	Explain how you would select chunk size empirically	29
Q2.3	Retrieval works for most documents and fails completely for a subset . . .	30
Q2.4	A wiki assistant returns the same navigation text for every question . . .	31
Q2.5	When is semantic chunking worth 40 times the ingestion cost?	32
Q2.6	How do you handle document updates and deletions without reindexing everything?	32
Q2.7	Why does adding the section heading to each chunk improve retrieval so much?	33
3	Embeddings and Vector Database Architecture	35
3.1	What an Embedding Represents	35
3.2	Dense and Sparse Representations	36
3.3	Similarity Measures	36
3.4	Vector Indexing and Approximate Search	37
3.5	Metadata Filtering and the Pre-filter Problem	38
3.6	Designing the pgvector Schema	40
3.7	Implementing Search With Correct Filtering	41
3.8	Testing Semantic Search Through the User Interface	42
3.9	Capacity Planning	43
3.10	Chapter Summary	44
3.11	Interview Questions and Answers	44
Q3.1	Design the storage layer for a multi-tenant RAG platform with 2,000 tenants	44
Q3.2	Explain HNSW well enough that I could implement search over it	45
Q3.3	Search returns nothing for a specific customer, yet their documents are indexed	46
Q3.4	Recall dropped after migrating from exhaustive search to HNSW	47
Q3.5	Would you choose pgvector or a dedicated vector database?	48
Q3.6	How do you upgrade the embedding model with zero downtime?	48
Q3.7	Why do similarity scores from different models mean different things? . .	49
4	Building a Complete Basic RAG System	51
4.1	The Retriever as an Interface, Not a Function	51
4.2	Context Construction	52
4.3	Prompt Engineering for Grounded Generation	54
4.4	Source Attribution That Users Can Verify	55
4.5	Conversation-Aware Retrieval	56
4.6	The Complete Pipeline	58
4.7	Testing the Complete System Through the User Interface	61
4.8	Chapter Summary	62
4.9	Interview Questions and Answers	63
Q4.1	Design a conversational RAG service with a strict 2 second p95	63
Q4.2	Citations point at the wrong sources	64
Q4.3	Why does placing the best chunk in the middle of the prompt hurt accuracy? .	64
Q4.4	Answers degrade only for long conversations	65
Q4.5	Should retrieval failure produce a refusal or a best effort answer?	66
Q4.6	How do you version prompts and roll them out safely?	66

Q4.7 Walk me through everything that happens between the user pressing enter and the first token appearing	67
Part II Advanced RAG	69
5 Advanced Retrieval and Hybrid RAG	71
5.1 Why Dense Retrieval Plateaus	71
5.2 BM25 and Lexical Retrieval	72
5.3 Hybrid Retrieval and Reciprocal Rank Fusion	72
5.4 Reranking	74
5.5 Query Transformation	77
5.6 Contextual Compression and Parent Child Retrieval	77
5.7 Corrective RAG and Adaptive RAG	78
5.8 Testing Advanced Retrieval Through the User Interface	81
5.9 Chapter Summary	83
5.10 Interview Questions and Answers	83
Q5.1 Design a hybrid retrieval service that stays within a 400 millisecond retrieval budget	83
Q5.2 Why fuse ranks instead of normalizing and adding scores?	84
Q5.3 Hybrid retrieval performs worse than dense alone	84
Q5.4 A legal research tool misses relevant precedents that a paralegal finds in seconds	85
Q5.5 When would you skip reranking entirely?	86
Q5.6 Design the query classifier that drives adaptive routing	87
Q5.7 Explain corrective RAG and where it belongs relative to answer verification	87
6 RAG Evaluation, Observability, and Debugging	89
6.1 Why a Single Accuracy Number Is Useless	89
6.2 Retrieval Metrics	89
6.3 Generation Metrics	92
6.4 Building a Golden Dataset That Survives	94
6.5 Online Monitoring	94
6.6 Failure Analysis	98
6.7 Testing the Evaluation Dashboard Through the User Interface	98
6.8 Chapter Summary	100
6.9 Interview Questions and Answers	100
Q6.1 Design the evaluation system for a RAG platform serving 40 teams	100
Q6.2 How do you validate that your LLM judge is trustworthy?	101
Q6.3 Offline metrics improved and users complained	102
Q6.4 Retrieval metrics look excellent and answers are still wrong	102
Q6.5 How much evaluation is enough before shipping?	103
Q6.6 Design the observability stack for a RAG system	104
Q6.7 Explain why claim level faithfulness beats paragraph level scoring	105
Part III Knowledge Graphs and GraphRAG	106
7 Knowledge Graph Fundamentals	107

7.1	Why Graphs Solve What Vectors Cannot	107
7.2	The Property Graph Model	107
7.3	Graph Modelling	108
7.4	Cypher for Traversal	109
7.5	Building a Knowledge Graph From Documents	110
7.6	Entity Resolution	114
7.7	Testing the Graph Builder Through the User Interface	116
7.8	Chapter Summary	117
7.9	Interview Questions and Answers	117
Q7.1	Design the knowledge graph layer for an enterprise with 200,000 documents	117
Q7.2	Why is a graph database faster than SQL for multi-hop queries?	118
Q7.3	The extracted graph is fragmented and traversals return nothing	119
Q7.4	An LLM built a graph with 340 relationship types	120
Q7.5	When is a knowledge graph not worth building?	121
Q7.6	How do you keep the graph synchronized with changing documents?	121
Q7.7	How do you prevent Cypher injection when relationship types come from an LLM?	122
8	GraphRAG and Hybrid Graph Retrieval	125
8.1	Where Vector-Only RAG Runs Out	125
8.2	Local Search, Entity-Centric Retrieval	126
8.3	Global Search, Community Detection and Summarization	129
8.4	Hybrid Retrieval and Context Fusion	129
8.5	Comparing Four Architectures	132
8.6	Testing Hybrid GraphRAG Through the User Interface	134
8.7	Chapter Summary	134
8.8	Interview Questions and Answers	136
Q8.1	Design a GraphRAG system for supply chain risk analysis	136
Q8.2	Explain local search and global search, and when each fails	137
Q8.3	GraphRAG returns empty context for half of all queries	137
Q8.4	A three hop query brought down the production Neo4j instance	138
Q8.5	Hybrid GraphRAG or advanced vector RAG for a customer support as- sistant?	139
Q8.6	How do you keep community summaries fresh without full recomputation?	140
Q8.7	Why does concatenating graph and vector context often make answers worse?	141
Part IV	Agentic RAG with LangGraph	142
9	Agentic GraphRAG and Multi-Hop Reasoning	143
9.1	From Pipeline to Policy	143
9.2	LangGraph Fundamentals	144
9.3	Designing the Node Set	146
9.4	Building the Graph	150
9.5	Human in the Loop	152
9.6	Testing the Agent Through the User Interface	152
9.7	Chapter Summary	154

9.8	Interview Questions and Answers	154
Q9.1	Design an agentic research assistant that answers multi-hop questions . .	154
Q9.2	Explain LangGraph state reducers and why they matter	155
Q9.3	The agent loops until the cap on most queries	156
Q9.4	An agent gave a different answer to the same question twice in a row . .	156
Q9.5	Fixed pipeline, router, or full agent?	157
Q9.6	How do you test an agentic workflow?	158
Q9.7	Where does human-in-the-loop belong, and how do you implement it with- out blocking?	159
Part V Production System Design and Interviews		161
10 Production RAG and GraphRAG System Design Interviews		163
10.1	The First Ten Minutes Decide the Interview	163
10.2	Scalability	164
10.3	Reliability	164
10.4	Security	168
10.5	Cost Optimization	170
10.6	Six Interview Scenarios	171
10.7	Testing Production Patterns Through the User Interface	173
10.8	Chapter Summary	174
10.9	Interview Questions and Answers	174
Q10.1	Design a RAG platform serving 50 million documents and 2,000 queries per second	174
Q10.2	Your p95 latency doubled overnight with no deployment	175
Q10.3	The board asks why the RAG project costs three times its forecast . . .	176
Q10.4	How do you enforce document-level access control in retrieval?	177
Q10.5	Local models or hosted API models for an enterprise deployment?	177
Q10.6	Design the rollout plan for replacing a keyword search system with RAG	178
Q10.7	Walk me through everything you would monitor, and what each alert means	179
Part VI Capstone, Glossary, and References		181
A Capstone, A Production-Style Agentic GraphRAG System		183
A.1	Target Architecture	183
A.2	Build Order and Acceptance Criteria	185
B Glossary		187
C References		191
Index		193

List of Figures

1.1	The four stage RAG reference architecture. The dashed boundary separates the offline indexing path, which tolerates high latency, from the online serving path, which does not.	6
1.2	Request sequence for a single question in RAG v1. The two Ollama calls dominate the latency budget, and the retrieval loop is pure in-process computation.	12
2.1	The ingestion pipeline with four chunking strategies branching from a shared parse and normalize stage. Parent child chunking writes to two stores, since children carry the vectors and parents carry the text.	23
2.2	Ingestion sequence for a single uploaded document. Note that the embedding call is batched once per strategy rather than once per chunk, which is the difference between a 400 millisecond and a 12 second ingest.	28
3.1	Vector database architecture for a multi-tenant RAG system. Filtering happens before the ANN traversal through a partial index, which avoids the post-filter recall collapse described below.	39
3.2	Query path through the vector store. Note that <code>SET LOCAL</code> runs inside the same transaction as the search, which is what makes per-query recall tuning and row level security work together.	43
4.1	Complete basic RAG pipeline with the four reliability gates highlighted. Each gate converts a silent failure into an observable event.	58
4.2	End to end sequence for one conversational turn, including the condensation short-cut and the Gate 3 regeneration loop.	62
5.1	Advanced retrieval architecture. The query classifier at the top decides which transformation and which retrieval branches run, which is what keeps the cost of these techniques bounded.	78
5.2	Hybrid retrieval sequence with concurrent branches and second stage reranking. Concurrency is what keeps hybrid retrieval within the same latency budget as dense retrieval alone.	82
6.1	Evaluation architecture. Offline evaluation gates deployment, online monitoring detects drift, and failure analysis feeds both back into the golden set.	95
6.2	Evaluation run sequence for one golden case. The judge calls are batched and run asynchronously in production sampling, which is what keeps evaluation affordable.	99
7.1	Knowledge graph construction pipeline. Schema constraint, entity resolution, and confidence filtering are the three stages that separate a usable graph from an un-queryable one.	111

7.2	Graph construction sequence for one document. The validator runs before any write, so a schema violation never reaches Neo4j.	117
8.1	Hybrid GraphRAG architecture. The router selects among four modes, and unified reranking merges graph and vector evidence into a single budgeted context rather than concatenating two.	133
8.2	Hybrid GraphRAG request sequence. Vector and graph retrieval run concurrently, and the graph-guided expansion issues a second vector query only when traversal discovered entities the question never named.	135
9.1	Agentic GraphRAG workflow. Solid edges are unconditional transitions, dashed edges are routing decisions, and the red edge is the bounded retrieval cycle that makes multi-hop reasoning possible.	147
9.2	Agent execution sequence for a two-hop question. Each node writes a checkpoint, which is what makes the run resumable and the trace complete.	153
10.1	Production reference architecture. The three planes scale independently, and the dashed boundaries mark the failure domains that determine blast radius.	165
10.2	Request lifecycle in the production architecture, showing where each reliability and security control applies.	173
A.1	Capstone system architecture. Every element traces to a chapter, and the numbered annotations mark the four gates that make failure observable.	184

List of Tables

1.1	Long-context prompting compared with retrieval augmented generation across the five dimensions that interviewers weigh.	4
1.2	Operational contrast between the indexing plane and the serving plane.	6
1.3	The six failure modes of naive RAG, each mapped to the chapter that resolves it. .	7
1.4	Directional impact of common design levers. “Up” means the property improves, and “Down” means it worsens.	9
2.1	Loader selection by format, and the specific signal that naive extraction discards. .	20
2.2	The metadata fields worth extracting at ingestion, with the query time capability each one unlocks.	22
2.3	Chunking strategy comparison. Ingest cost is relative to fixed size chunking on the same corpus.	24
3.1	Dense and sparse representations differ most in the queries they fail on, which is why combining them in Chapter 5 raises recall substantially.	36
3.2	Index selection for pgvector. Latency figures assume 768 dimensional vectors on commodity server hardware and should be re-measured on your own workload. . .	38
3.3	Capacity estimates at 768 dimensions in float32, assuming 50 chunks per document. Text and metadata storage is excluded and often doubles the total.	44
4.1	Prompt elements that measurably change behaviour on small local models. Explicit failure tokens outperform natural language refusals because they are checkable in code.	55
5.1	Retrieval technique selection by query class. No single retriever wins across the table, which is the argument for hybrid retrieval.	71
5.2	Ranking model comparison. The standard production pattern retrieves 30 candidates with a bi-encoder and reranks to 5 with a cross encoder.	75
6.1	The two by two that a single accuracy number destroys. Each cell demands a different engineering response.	90
6.2	Golden dataset design. Every row on the right costs more to produce, and every one of them prevents a specific class of blind spot.	95
6.3	Online signals that work without ground truth labels. The behavioural signals in the lower half often detect quality regressions before the model based ones do. . . .	96
7.1	Query classes where graph retrieval beats vector retrieval. The pattern is consistent: joins and paths favour graphs, contiguous passages favour vectors.	108
7.2	Graph modelling failure modes and their corrections. Every one of these appears in graphs built by naive LLM extraction.	109

8.1	Traversal cost by depth. Two hops answers most production questions, three is the practical ceiling for interactive latency, and beyond that you need shortest path rather than exhaustive traversal.	126
8.2	The three retrieval modes in a hybrid GraphRAG system. Routing between them is the design decision that determines both cost and quality.	130
8.3	Architecture comparison. Hybrid GraphRAG is not universally best, and defending when it is not is what separates a strong interview answer from a weak one.	133
9.1	What agency buys and what it costs. Every row on the right is a design obligation, not an automatic benefit.	143
9.2	Node responsibilities. The value of each node is best understood through what breaks in its absence.	146
10.1	The eight requirements to establish before drawing anything. Each one changes a specific architectural decision, so name the connection when you ask.	164
10.2	Scalability patterns and the specific failure each one prevents. Naming the failure is what makes the pattern an argument rather than a list item.	165
10.3	Reliability patterns with the RAG specific caveat that generic system design answers usually miss.	166
10.4	Security controls for enterprise RAG. The right column is what turns a checklist into an answer.	169
10.5	Cost levers ordered roughly by return on effort. The first four cost nothing in quality, which is why they come first.	170
A.1	Capstone technology stack and the responsibility of each layer.	183

PART I

RAG Foundations

Everything in this book rests on the four ideas established here. A retrieval augmented system is not a model with a database bolted on. It is a distributed system with an offline indexing plane and an online serving plane, and almost every production defect traces back to whichever plane you neglected.

These four chapters build that system from nothing. Chapter 1 fixes the mental model and the vocabulary interviewers expect. Chapter 2 shows why ingestion sets a ceiling that no downstream tuning can raise. Chapter 3 turns retrieval from a black box into an engineering discipline with a schema, an index, and a filtering strategy. Chapter 4 assembles the pieces into a system with explicit contracts, deterministic context construction, and failure that is visible rather than silent.

Read this part in order. Every later chapter assumes the four stage model and the retriever contract defined here.

RAG Foundations and System Design Fundamentals

Subscribe → aiengineeringinsider.substack.com/subscribe

What This Chapter Delivers

This chapter establishes the mental model that every later chapter builds on. You will learn why large language models hallucinate, why context windows fail as a knowledge store, and how Retrieval Augmented Generation converts a closed-book model into an open-book system. You will then translate that understanding into system design language: functional requirements, non-functional requirements, and the accuracy, latency, scalability, and cost quadrilateral that interviewers probe. The chapter closes with a working RAG v1 lab and seven interview questions drawn from real production incidents.

1.1 Why Language Models Hallucinate

A large language model is a parametric function. Training compresses a corpus into weights, and inference samples tokens from a probability distribution conditioned on the prompt. That architecture produces two structural failure modes that no amount of prompt engineering fully removes.

1.1.1 Parametric Memory Is Lossy

During pretraining, the model observes a token sequence and adjusts weights to reduce next-token loss. Consequently, it stores statistical regularities rather than verbatim facts. A fact that appears once in the corpus barely moves the loss surface, so the model retains only a fuzzy impression of it. When you later ask about that fact, the model still produces fluent text, because fluency and factuality come from different parts of the objective. Fluency is directly optimized. Factuality is only optimized as a side effect of fluency on a factual corpus.

Hallucination

A hallucination is a model output that is syntactically well formed and semantically confident, yet unsupported by any verifiable source. Hallucinations divide into two families. **Intrinsic hallucination** contradicts the provided context. **Extrinsic hallucination** introduces claims absent from the provided context, and the model cannot verify them.

1.1.2 The Knowledge Cutoff Is Structural

Weights freeze at the end of training. Therefore, any event, policy change, price update, or internal document created afterward simply does not exist for the model. Fine-tuning appears to

solve this, but it does not scale. Fine-tuning costs GPU hours per update, it risks catastrophic forgetting, it cannot enforce access control per user, and it provides no citation trail. A bank cannot fine-tune a model every time a compliance memo changes.

1.1.3 Confidence Is Not Calibrated to Evidence

The decoder emits a probability distribution over the vocabulary, not a probability over truth. A model that is 92 percent confident in the next token says nothing about whether the resulting sentence is factual. In production this creates the most dangerous class of defect: an authoritative wrong answer that a user forwards to a customer.

! The Silent Failure Mode

Retrieval failure and generation failure look identical to the end user. Both produce a confident paragraph. Only a system that surfaces retrieved sources, similarity scores, and a groundedness signal lets an operator distinguish “the retriever found nothing” from “the retriever found the right chunk and the generator ignored it”. Chapter 6 builds that instrumentation.

1.2 Why the Context Window Is Not a Database

Modern models advertise context windows of 128,000 tokens and beyond, which tempts engineers to skip retrieval and paste everything into the prompt. That approach breaks along four axes.

Axis	Long-context stuffing	Retrieval augmented generation
Cost	Grows linearly with corpus size on every request, because you resend the entire corpus each time.	Grows with K retrieved chunks, which stays roughly constant regardless of corpus size.
Latency	Attention cost grows quadratically with sequence length in the prefill phase.	Retrieval adds 10 to 60 milliseconds, then generation sees a short prompt.
Accuracy	Degrades through the lost-in-the-middle effect, where the model attends weakly to tokens far from the prompt edges.	Improves, because the top ranked chunks sit at the prompt edges where attention is strongest.
Freshness	Requires resending the corpus after every edit.	Requires reindexing only the changed documents.
Governance	No per-user filtering, since the whole corpus enters the prompt.	Metadata filters and row level security restrict retrieval per tenant and per role.

Table 1.1: Long-context prompting compared with retrieval augmented generation across the five dimensions that interviewers weigh.

Understanding these results and requirements provides clear operational context for the system design.

★ Retrieval Is a Compression Strategy

Treat retrieval as lossy compression of the corpus, conditioned on the query. The retriever answers one question: which 2,000 tokens out of 200 million best support this specific request? Framing RAG this way makes every downstream decision, including chunk size, embedding model, and reranking, a compression quality decision rather than a plumbing decision.

1.3 The Four Stages of a RAG System

Every RAG system, from a weekend prototype to a platform serving ten million documents, decomposes into four stages. Two stages run offline. Two stages run online.

1.3.1 Stage One, Indexing

Indexing ingests source documents, normalizes them into plain text, splits them into retrievable units, converts each unit into a vector, and writes the vector plus its metadata into a store. This stage runs as a batch or streaming job. It tolerates seconds or minutes of latency, so you can afford expensive operations here such as layout aware PDF parsing, table extraction, and entity resolution.

1.3.2 Stage Two, Retrieval

Retrieval embeds the incoming query with the same model used for documents, then performs approximate nearest neighbour search against the index. It returns the top K candidates ranked by similarity. This stage runs online and sits directly on the user latency path, so its budget is measured in tens of milliseconds.

1.3.3 Stage Three, Augmentation

Augmentation assembles retrieved chunks into a prompt. Despite its simplicity, this stage causes a large share of production defects. Ordering matters, because attention favours the prompt edges. Deduplication matters, because near duplicate chunks waste budget. Delimiting matters, because a model that cannot tell where a source begins cannot cite it. Token budgeting matters, because an overflowing prompt silently truncates the final chunk.

1.3.4 Stage Four, Generation

Generation calls the language model with the augmented prompt and a system instruction that constrains it to the supplied evidence. A well designed generation stage also emits structure: the answer, the source identifiers, and an abstention signal when evidence is insufficient.

1.4 Offline Indexing Compared With Online Serving

Interviewers listen for whether you separate these two planes, because the separation drives almost every operational decision.

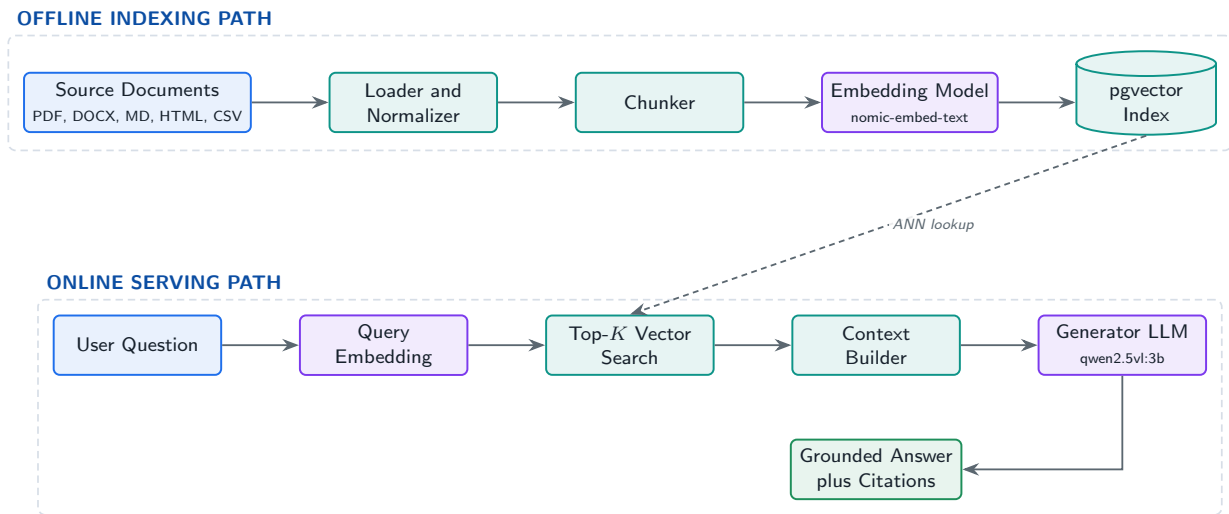


Figure 1.1: The four stage RAG reference architecture. The dashed boundary separates the offline indexing path, which tolerates high latency, from the online serving path, which does not.

Property	Offline indexing	Online serving
Latency budget	Minutes to hours per batch.	200 to 2,000 milliseconds end to end.
Failure handling	Retry the whole job, idempotently.	Degrade gracefully, never retry blindly.
Scaling unit	Worker pool sized by document backlog.	Replica count sized by queries per second.
Cost driver	Embedding calls, parsing compute.	Generation tokens, ANN search.
Correct trigger	Document created, updated, or deleted.	User request arrives.
Idempotency	Mandatory, keyed on a content hash.	Not applicable, but caching is.
Typical bug	Duplicate chunks after a partial rerun.	Empty retrieval returning a confident guess.

Table 1.2: Operational contrast between the indexing plane and the serving plane.

Understanding these results and requirements provides clear operational context for the system design.

✓ Key Every Chunk on a Content Hash

Compute `sha256(normalized.text + source.uri + chunk.index)` and store it as a unique column. Ingestion then becomes an upsert rather than an insert. Consequently, a job that crashes halfway through and restarts produces no duplicates, and duplicate chunks are the single most common cause of a retriever returning five copies of the same paragraph.

1.5 Naive RAG and Its Failure Surface

Naive RAG is the architecture in Figure 1.1 with no reranking, no query rewriting, and no evaluation. It is the correct starting point, because it establishes a measurable baseline. However, you must be able to name its failure modes on demand.

Failure	Symptom	Root cause and chapter that fixes it
Semantic gap	User asks about “termination clause”, corpus says “notice of cancellation”.	Vocabulary mismatch in dense space. Hybrid retrieval, Chapter 5.
Lost context	Retrieved chunk starts mid sentence and lacks the section heading.	Naive fixed size chunking. Structure aware chunking, Chapter 2.
Multi-hop miss	Question needs two documents joined by an entity.	Single shot top- K cannot join. Graph retrieval, Chapter 8.
Distractor poisoning	A high scoring but irrelevant chunk dominates the answer.	No reranking stage. Cross encoder rerank, Chapter 5.
Silent empty retrieval	Corpus lacks the answer, model invents one.	No abstention policy. Groundedness gate, Chapter 6.
Stale answer	Document changed, index did not.	No change data capture. Ingestion design, Chapter 10.

Table 1.3: The six failure modes of naive RAG, each mapped to the chapter that resolves it.

1.6 Framing RAG as a System Design Problem

An interviewer who asks “design a document question answering system” expects requirements before architecture. Therefore, open with a requirements pass rather than a diagram.

1.6.1 Functional Requirements

- Users upload documents in PDF, DOCX, Markdown, HTML, and CSV formats.
- The system answers natural language questions grounded in those documents.
- Every answer cites the source document, the page or section, and a retrieval score.

- The system abstains when the corpus contains no supporting evidence.
- Conversation history influences retrieval, so follow up questions resolve pronouns.
- Administrators reindex, delete, and audit documents.

1.6.2 Non-Functional Requirements

- **Latency:** p50 under 1.5 seconds and p95 under 4 seconds for the first token.
- **Accuracy:** faithfulness above 0.90 and answer relevance above 0.85 on a golden set.
- **Scalability:** 10 million chunks, 500 queries per second at peak.
- **Freshness:** a document edit becomes searchable within 5 minutes.
- **Availability:** 99.9 percent monthly, with graceful degradation rather than hard failure.
- **Security:** tenant isolation at the row level, plus prompt injection defence.
- **Cost:** under a defined cost per thousand queries, including embedding and generation.

★ Say the Numbers Out Loud

Candidates who state “low latency” score poorly. Candidates who state “p95 under 4 seconds, which gives the retriever 60 milliseconds, the reranker 150 milliseconds, and the generator the remaining 3.7 seconds” score well, because a latency budget that decomposes into per stage allocations proves you have built the system rather than read about it.

1.6.3 The Four Way Trade-off

Accuracy, Latency, Scalability, and Cost

These four properties form a constraint surface where improving one usually degrades another. Increasing K from 5 to 20 raises recall, and it also raises prompt tokens by four times, which raises cost and latency together. Adding a cross encoder reranker raises precision by 10 to 20 points, and it adds 100 to 300 milliseconds. Switching to a larger embedding model raises retrieval quality, and it raises index size, memory footprint, and reindexing cost. State the trade you are making and the metric you will watch, because interviewers grade the reasoning rather than the choice.

The following section and table detail the technical specifications and evaluation metrics.

1.7 Building RAG v1

The companion repository implements this chapter as an in-memory RAG with no external database. That choice is deliberate. It isolates the four stages so you can observe each one before Chapter 3 introduces pgvector.

Lab Code for This Chapter

Run the reference implementation from `chapter_01/rag_basic.py` in the companion repository at github.com/lamhosiagian/rag-graphrag-lab. Start the Streamlit interface with `streamlit run chapter_01/app.py`, then upload a document and ask a question.

Design lever	Accuracy	Latency	Cost	Scalability
Raise K from 5 to 20	Up	Down	Down	Neutral
Add cross encoder rerank	Strongly up	Down	Down	Down
Smaller chunk size	Up for precision	Neutral	Up for storage	Down
HNSW instead of exact search	Slightly down	Strongly up	Up	Strongly up
Semantic response cache	Neutral	Strongly up	Strongly up	Up
Larger generator model	Up	Down	Strongly down	Down
Query rewriting with a small model	Up	Slightly down	Slightly down	Neutral

Table 1.4: Directional impact of common design levers. “Up” means the property improves, and “Down” means it worsens.

1.7.1 The Retrieval Core

The reference implementation embeds each chunk once, stores the vectors in a Python list, and scores the query against every vector. Exhaustive search is correct at this scale and it makes the similarity computation explicit.

chapter_01/rag_basic.py, indexing and retrieval core

```

1 import numpy as np
2 from typing import Any, Dict, List
3
4
5 def build_simple_index(
6     chunks: List[str],
7     embeddings_model,
8 ) -> List[Dict[str, Any]]:
9     Embed every chunk once and hold the vectors in process memory.
10
11     Batching all chunks into a single embed_documents call matters. One
12     HTTP round trip to Ollama for 200 chunks costs roughly 400 ms, while
13     200 sequential calls cost roughly 12 s on the same hardware.
14
15     index: List[Dict[str, Any]] = []
16     embeddings = embeddings_model.embed_documents(chunks)
17     for chunk, vector in zip(chunks, embeddings):
18         index.append({content: chunk, embedding: np.asarray(vector)})
19     return index
20
21
22 def cosine_similarity(a: np.ndarray, b: np.ndarray) -> float:
23     Cosine similarity, which is the dot product of the L2 normalized vectors.
24     denominator = np.linalg.norm(a) * np.linalg.norm(b)
25     if denominator == 0.0:
26         return 0.0
27     return float(np.dot(a, b) / denominator)

```

```

28
29
30 def retrieve(
31     query: str,
32     index: List[Dict[str, Any]],
33     embeddings_model,
34     top_k: int = 3,
35     min_score: float = 0.0,
36 ) -> List[Dict[str, Any]]:
37     Score the query against every chunk and return the strongest matches.
38
39     The min_score floor is the abstention hook. When the best chunk falls
40     below the floor, the caller receives an empty list and must refuse to
41     answer rather than pass weak evidence to the generator.
42
43     query_vector = np.asarray(embeddings_model.embed_query(query))
44
45     scored = [
46         {
47             content: item[content],
48             score: cosine_similarity(query_vector, item[embedding]),
49         }
50         for item in index
51     ]
52     scored.sort(key=lambda row: row[score], reverse=True)
53     return [row for row in scored[:top_k] if row[score] >= min_score]

```

1.7.2 The Grounded Generation Prompt

The prompt is a contract. It tells the model what evidence it may use, what to do when the evidence runs out, and how to cite. Weak prompts produce confident fabrication, therefore treat the prompt as production code and version it.

chapter_01/rag_basic.py, grounded generation with an explicit abstention rule

```

1 GROUNDED_SYSTEM_PROMPT = You are a document question answering assistant.
2
3 RULES
4 1. Answer using ONLY the numbered context blocks below.
5 2. Cite every claim with the block number in square brackets, for example [2].
6 3. If the context does not contain the answer, reply exactly:
7     The provided documents do not contain enough information to answer this.
8 4. Never use knowledge from outside the context blocks.
9 5. Keep the answer under 200 words unless the question asks for a list.
10
11
12
13 def build_context(chunks: list[dict]) -> str:
14     Render retrieved chunks as numbered, delimited blocks.
15
16     Numbering is what makes citation possible. A model cannot cite a source
17     it cannot address, so an unnumbered blob of concatenated text guarantees
18     uncited answers no matter how strict the system prompt sounds.
19
20     blocks = []
21     for position, chunk in enumerate(chunks, start=1):

```

```

22     score = chunk.get(score, 0.0)
23     blocks.append(
24         f[{position}] (relevance {score:.3f})\n{chunk['content'].strip()}
25     )
26     return \n\n.join(blocks)
27
28
29 def generate_answer(query: str, context_chunks: list[dict], llm) -> str:
30     Call the generator with a grounded prompt, or abstain when evidence is empty.
31     if not context_chunks:
32         return (
33             The provided documents do not contain enough information
34             to answer this.
35         )
36
37     prompt = (
38         f{GROUNDED_SYSTEM_PROMPT}\n\n
39         fCONTEXT\n{build_context(context_chunks)}\n\n
40         fQUESTION\n{query}\n\n
41         fANSWER
42     )
43     response = llm.invoke(prompt)
44     return response.content if hasattr(response, content) else str(response)

```

! The Abstention Path Must Bypass the Model

Notice that the empty context branch returns a literal string rather than asking the model to abstain. A 3 billion parameter model asked to abstain still answers roughly 30 percent of the time, because instruction following degrades under distribution shift. Enforce refusal in code, then use the model only when evidence exists.

1.8 Testing RAG v1 Through the User Interface

The Streamlit application in [chapter_01/app.py](#) exposes each stage so you can attribute a bad answer to a specific component. Follow this sequence.

1. Start Ollama and confirm both models are pulled: `ollama pull qwen2.5v1:3b` and `ollama pull nomic-embed-text`.
2. Launch the interface with `streamlit run chapter_01/app.py` and open `http://localhost:8501`.
3. Upload [test_data/sample.md](#) from the repository, then press **Process Document**. The sidebar reports the chunk count, which verifies the indexing stage.
4. Ask a question whose answer appears verbatim in the document. Expand the **Sources** panel and confirm the top source contains the answer text. That verifies retrieval independently of generation.
5. Ask a question the document cannot answer, for example “what is the CEO’s home address”. The system must return the abstention string. That verifies the groundedness gate.
6. Ask a paraphrased question that avoids the document’s vocabulary. Observe the similarity scores drop. That reproduces the semantic gap failure from Table 1.3, which Chapter 5 repairs with hybrid retrieval.

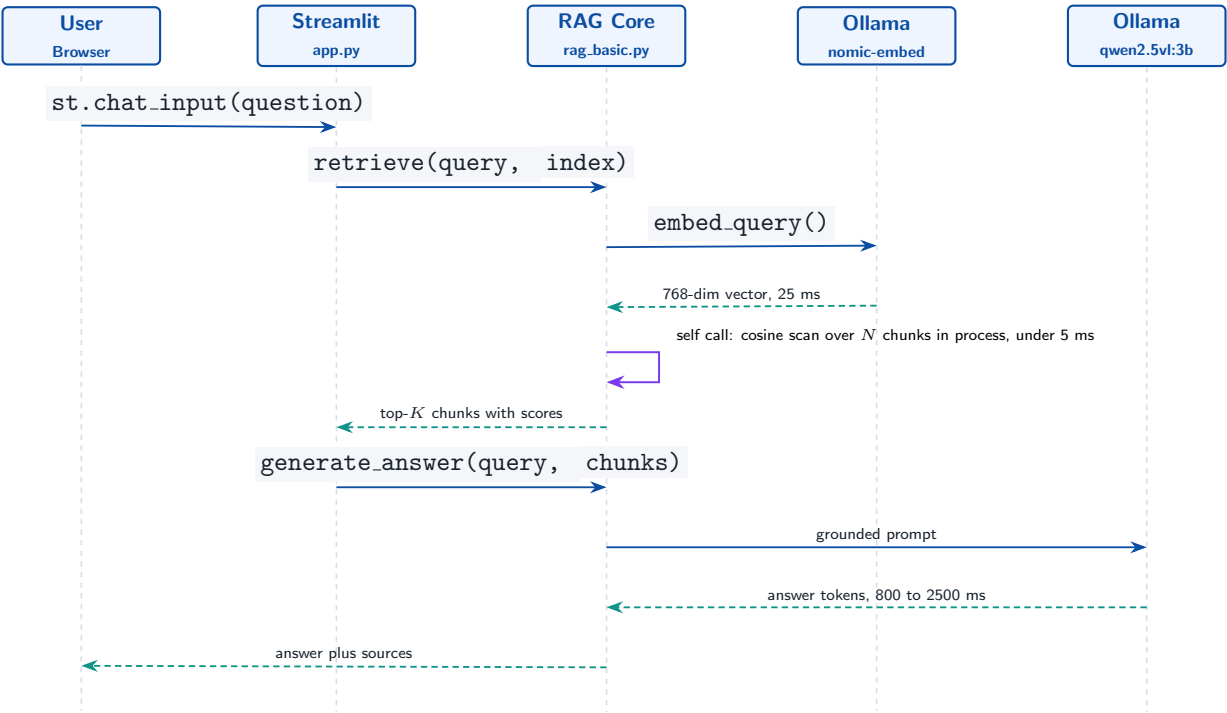


Figure 1.2: Request sequence for a single question in RAG v1. The two Ollama calls dominate the latency budget, and the retrieval loop is pure in-process computation.

The diagram above illustrates the complete architectural pipeline and state interactions.

Enterprise Wiki Rollout, 40,000 Pages

A platform team shipped naive RAG over an internal wiki. Week one satisfaction reached 71 percent. Week three it fell to 44 percent. Investigation showed three causes. First, the wiki contained six versions of an onboarding page and the retriever returned all six, so answers mixed obsolete policy with current policy. Second, chunk boundaries split tables, so numeric answers lost their column headers. Third, deleted pages remained in the index because deletion events were never wired into ingestion. None of these were model problems. All three were indexing plane problems, which is why the team’s first fix, upgrading the generator, produced no measurable improvement.

1.9 Chapter Summary

RAG converts a closed-book model into an open-book system by injecting retrieved evidence at inference time. It solves the knowledge cutoff, it enables citation, and it supports access control, none of which fine-tuning provides economically. The architecture splits into an offline indexing plane and an online serving plane, and almost every production defect traces to the plane you neglected. Design conversations should begin with functional and non-functional requirements, then move to a latency budget decomposed per stage, and finally to the accuracy, latency, scalability, and cost trade-off you are consciously accepting.

1.10 Interview Questions and Answers

Q1.1 Design a document question answering system for 50,000 internal documents

System Design

Q. *Walk me through your architecture, and justify the components you include as well as the ones you leave out.*

Answer. Begin with requirements rather than components. Ask for document count and growth rate, average document size, query volume at peak, acceptable p95 latency, freshness requirement, and whether answers must be auditable. Assume 50,000 documents averaging 12 pages, which yields roughly 2 million chunks at 400 tokens each, plus 50 queries per second at peak and a 3 second p95 target.

Split the design into two planes. The indexing plane runs a worker pool consuming from a queue fed by document change events. Each worker parses the document with a layout aware parser, normalizes whitespace and encoding, chunks recursively along structural boundaries, computes a content hash per chunk, embeds in batches of 64, and upserts into pgvector keyed on that hash. Idempotency through the hash makes retries safe. Size the pool by backlog depth rather than by document count.

The serving plane accepts a question, embeds it with the same model, runs an HNSW search filtered by tenant and by access control list, retrieves the top 20, reranks to the top 5 with a cross encoder, builds a numbered context, and calls the generator with a grounded prompt. Return the answer plus source identifiers and scores.

Justify the omissions explicitly. Skip a graph database initially, because 90 percent of queries are single hop lookups and Neo4j adds an ingestion pipeline you would have to operate. Skip fine-tuning, because it cannot enforce per user access control and it provides no citation. Skip a dedicated vector database, because pgvector inside the existing PostgreSQL cluster removes an operational dependency and handles 2 million vectors comfortably. Add each of these later, gated on a metric: add the graph when multi-hop queries exceed 15 percent of traffic, and add a dedicated vector store when index size exceeds available memory.

Close with the latency budget. Embedding costs 25 milliseconds, HNSW search costs 15 milliseconds, reranking costs 180 milliseconds, prompt assembly costs 5 milliseconds, and generation consumes the remaining 2.7 seconds. Generation dominates, therefore streaming the first token matters more than shaving milliseconds off retrieval.

Q1.2 Answers are confident and wrong. How do you localize the fault?

Production

Debugging

Q. *A user reports that the assistant invents policy details. You have logs, the index, and the ability to replay requests. Where do you look, and in what order?*

Answer. Localize before you fix. The pipeline has four stages, so bisect them.

First, test retrieval in isolation. Take the failing question, run only the retriever, and inspect the top K chunks by hand. Two outcomes follow. If the correct chunk is absent, the fault lies upstream in indexing or retrieval. If the correct chunk is present, the fault lies in

augmentation or generation.

When the chunk is absent, check indexing next. Query the store for any chunk containing a distinctive phrase from the source document. If nothing matches, the document never indexed, so inspect the ingestion job for a parse failure, an unsupported format, or a silently swallowed exception. If the chunk exists but ranks low, you have a retrieval quality problem: check whether the query and document embeddings came from the same model version, check whether the chunk is so long that its embedding is diluted, and check whether a metadata filter excluded it.

When the chunk is present but the answer is still wrong, inspect the assembled prompt verbatim. Common causes include the chunk being truncated by a token budget, near duplicate chunks crowding out the relevant one, the relevant chunk sitting in the middle of a long context where attention is weakest, and a system prompt that permits outside knowledge.

Add permanent instrumentation so the next incident takes minutes rather than days. Log the query, the retrieved chunk identifiers with scores, the exact prompt hash, the model version, and a groundedness score computed by a second model. Then build the failure taxonomy from Table 1.3 into a dashboard so each incident is classified rather than rediscovered.

Q1.3 Why not simply use a one million token context window?

Technical Depth

Q. *Your leadership asks why the team is building retrieval infrastructure when the newest model accepts a million tokens.*

Answer. Answer on cost, latency, accuracy, and governance rather than on preference.

On cost, prompt tokens are billed per request. A 200,000 token corpus resent on every one of 100,000 daily queries costs 20 billion input tokens per day. Retrieval sends roughly 2,500 tokens per query, which is 250 million tokens per day, an eighty fold reduction. Even with prompt caching, the cache must be rebuilt whenever the corpus changes and cache storage itself is billed.

On latency, prefill attention scales quadratically with sequence length before key value caching amortizes it. Time to first token on a very long prompt runs into tens of seconds, which breaks any interactive product.

On accuracy, the lost-in-the-middle effect is well documented: retrieval accuracy on long contexts degrades sharply for information placed away from the prompt boundaries, and it degrades further as distractor density rises. Retrieval improves accuracy precisely because it removes distractors before the model sees them.

On governance, a stuffed context has no access control. Every user sees every document, and you cannot filter by tenant, role, or classification. Retrieval applies filters at query time, so a contractor and an executive issue the same question and receive different evidence.

Concede the legitimate case. Long context excels when the corpus is genuinely small, when the task requires holistic reasoning across an entire document such as summarizing a contract, and when latency is not interactive. Many mature systems combine both: retrieval selects

the relevant documents, then a long context model reasons over the complete text of those few documents.

Q1.4 The customer support bot regressed after a model upgrade

Case Study

Q. *Retrieval quality dropped 30 points immediately after the team upgraded the embedding model. Nothing else changed. Diagnose it.*

Answer. This is an embedding version skew incident, and it is among the most common outages in production RAG.

The mechanism is straightforward. Document vectors in the index were produced by the old model. After the upgrade, query vectors come from the new model. The two models place text in different vector spaces, therefore cosine similarity between a new query vector and an old document vector is close to noise. Scores still look plausible because cosine similarity always returns a number, which is why the failure is silent rather than loud.

Confirm the diagnosis in minutes. Embed a document chunk with the new model, compare it against its own stored vector, and observe a similarity far below 1.0 when it should be exactly 1.0. Also check the vector dimension: a dimension mismatch would have raised an error, and its absence tells you both models share a dimension, which is exactly the case that fails silently.

Remediate in two steps. Immediately roll the query path back to the old model, which restores service without touching the index. Then reindex the corpus with the new model in a shadow table, validate on a golden set, and cut over atomically by swapping a table name or an index alias.

Prevent recurrence structurally. Store the embedding model name and version as a column on every chunk. Refuse at query time to search vectors whose model version differs from the query encoder. Add a startup assertion that compares the configured model against the distinct model versions present in the index. Finally, treat the embedding model as part of the index schema, so changing it requires a migration rather than a configuration edit.

Q1.5 How do you choose K , and what breaks at the extremes?

Trade-offs

Q. *Your retriever returns the top K chunks. Explain how you would select K empirically and what fails when K is too small or too large.*

Answer. Treat K as a tuned parameter measured against a golden dataset rather than a constant copied from a tutorial.

When K is too small, recall collapses. The answer sometimes lives in the sixth ranked chunk, and the generator never sees it, so the system abstains or fabricates. Multi-part questions suffer most, because they need evidence from several chunks simultaneously.

When K is too large, three problems compound. Precision falls, because low ranked chunks are distractors and the generator has no reliable way to ignore them. Cost rises linearly

with prompt tokens. Latency rises, and the relevant chunk drifts toward the middle of the prompt where attention is weakest.

Measure rather than guess. Build a golden set of 100 to 300 question and answer pairs with labelled relevant chunks. Sweep K from 1 to 30 and plot recall at K , precision at K , faithfulness, and p95 latency on a single chart. Recall typically saturates between 10 and 20 while precision falls monotonically, so the useful operating point sits where the recall curve flattens.

Then decouple the two roles of K . Retrieve a wide candidate set, for example 30, so recall saturates. Rerank those candidates with a cross encoder and pass only the top 4 or 5 to the generator, which restores precision. This two stage pattern reliably outperforms any single K because retrieval optimizes recall and reranking optimizes precision.

Finally, make K adaptive. Classify the query first. A factual lookup needs 3 chunks. A comparison across entities needs 10. A summarization request may need 20. Routing K by query type reduces average cost while raising accuracy on the hard tail.

Q1.6 Design the abstention policy and defend it against product pressure *Production*

Q. *Product management complains that the assistant refuses too often. Engineering complains that lowering the threshold causes hallucination. Resolve this.*

Answer. Convert an opinion argument into a measured one by defining the two error types and their asymmetric costs.

A false answer occurs when the system responds without adequate evidence. A false refusal occurs when the system abstains although evidence exists. In a legal, medical, or financial domain a false answer costs far more than a false refusal, so the operating point should sit conservatively. In a general productivity assistant the costs are closer, so the operating point can relax.

Build abstention from three independent signals rather than one threshold. First, a retrieval floor, where the top similarity score must exceed a calibrated value. Calibrate it by plotting the score distribution for answerable and unanswerable questions on the golden set and choosing the crossing point rather than a round number. Second, a coverage check, where a small model judges whether the retrieved context actually addresses the question. Third, a post generation groundedness check, where every claim in the answer must map to a context span, and a failure triggers refusal.

Improve the refusal itself, because most product complaints concern the message rather than the decision. Instead of a bare refusal, return the closest documents found, state which part of the question lacked evidence, and offer a narrower reformulation. Refusal quality often resolves the dispute without moving the threshold at all.

Instrument the decision so the argument stays empirical. Log every abstention with its triggering signal, sample them weekly, and label whether the corpus truly lacked the answer. If most abstentions are correct, the retriever is fine and the corpus has gaps, which is a content problem. If most are incorrect, the retriever is under-recalling, which is an engineering problem and threshold tuning is the wrong fix.

Q1.7 Take this design from 50,000 documents to 10 million*Scalability*

Q. *Same product, two hundred times the corpus. What changes, and what stays the same?*

Answer. The four stage architecture stays. The implementation of each stage changes.

Indexing moves from a single process to a horizontally scaled worker pool consuming a durable queue such as Kafka or SQS. Introduce a document level lock keyed on the source identifier so concurrent updates to the same document do not interleave chunks. Batch embedding calls and run several workers against a pool of embedding replicas, since embedding throughput, not parsing, becomes the bottleneck at this scale.

Storage crosses the memory threshold. Roughly 400 million chunks at 768 dimensions in float32 occupies about 1.2 terabytes, which no single node holds in memory. Three responses apply. Shard by tenant or by document collection so each shard fits in memory. Quantize vectors, since scalar quantization to int8 cuts the footprint fourfold with a small recall loss, and product quantization cuts it further at a larger loss. Adopt a two tier index where a small in-memory index over centroids narrows the search before a disk resident index resolves it.

Retrieval changes from a single query to a scatter gather across shards, followed by a merge on normalized scores. Latency now depends on the slowest shard, therefore add hedged requests and a per shard timeout that returns partial results rather than failing the query.

Serving adds caching layers. An exact response cache keyed on the normalized query catches repeats. A semantic cache keyed on the query embedding catches paraphrases. An embedding cache keyed on the query string avoids re-embedding common queries. Together these commonly absorb 30 to 50 percent of traffic in a support workload.

Freshness needs change data capture rather than full reindexing, since a full pass over 10 million documents takes days. Emit create, update, and delete events from the source system, and process deletes eagerly because stale answers from deleted documents create compliance exposure.

What stays the same is the contract: same four stages, same idempotent content hash, same grounded prompt, same golden set. Preserving the contract is what lets you replace each implementation independently.

Document Ingestion, Parsing, and Chunking

Subscribe → aiengineeringinsider.substack.com/subscribe

What This Chapter Delivers

Retrieval quality is bounded by chunk quality, therefore this chapter treats ingestion as a first class subsystem rather than a preprocessing script. You will learn how loaders differ across PDF, DOCX, Markdown, HTML, and CSV, why normalization silently destroys retrieval signal when done carelessly, and how metadata extraction unlocks filtering that no embedding model can replace. You will then work through four chunking strategies, measure their trade-offs, and build the comparison lab that quantifies the impact of chunk size and overlap on retrieval.

2.1 Ingestion Is Where Retrieval Quality Is Decided

Engineers routinely spend weeks tuning retrievers on top of a corpus that ingestion already corrupted. Consider the arithmetic. If the parser drops table structure, then no embedding model can recover the association between a number and its column header. If the chunker splits a definition from its subject, then no reranker can reunite them. Ingestion sets a ceiling, and every downstream stage operates below that ceiling.

★ The Ingestion Ceiling

Define the ingestion ceiling as the fraction of golden questions whose answer is fully contained inside at least one chunk. Measure it before you tune anything else. A corpus with a 68 percent ceiling cannot exceed 68 percent recall no matter how sophisticated the retriever becomes, and teams commonly discover their real problem here after months of retriever tuning.

2.2 Document Loaders by Format

Each format carries different structure, and each loses different information under naive extraction.

2.2.1 PDF

PDF stores drawing instructions rather than a document tree. Text extraction therefore reconstructs reading order from glyph positions, which fails predictably on multi column layouts, footnotes, headers, and tables. Three tiers of tooling exist.

- **Stream extractors** such as `pypdf` read the text layer directly. They are fast and dependency light, and they lose layout completely.
- **Layout aware parsers** such as `pdfplumber` or `unstructured` reconstruct columns, headings, and table cells from coordinates. They cost 10 to 50 times more compute, and they preserve the structure that answers numeric questions.
- **Vision parsers** run a multimodal model over rendered pages. They handle scanned documents and complex layouts, and they cost the most.

Route by document rather than choosing globally. Detect whether a page carries a text layer, and fall back to OCR only for the pages that lack one.

2.2.2 DOCX, Markdown, and HTML

These formats carry an explicit structure tree, which is a gift you should not discard. Headings define natural chunk boundaries. Lists define atomic units. Tables carry a header row you can repeat into every row derived chunk. Consequently, structure aware chunking on these formats routinely outperforms fixed size chunking by 15 to 25 points of recall.

2.2.3 CSV and Tabular Data

Tables break the assumption that a chunk is a paragraph. Two strategies dominate. Row serialization converts each row into a sentence that repeats the column names, which makes each row independently retrievable. Table summarization generates a natural language description of the whole table and indexes that alongside the raw rows, which supports aggregate questions such as “which region grew fastest”.

Format	Structure available	What naive loading loses	Recommended handling
PDF, digital	Coordinates only	Columns, tables, headings, footnotes	Layout aware parser, table extraction to Markdown
PDF, scanned	None	Everything	OCR per page, then layout reconstruction
DOCX	Full tree	Heading hierarchy, styles, comments	Walk the tree, chunk on heading boundaries
Markdown	Full tree	Heading hierarchy, code fences	Split on headings, keep code fences intact
HTML	Full DOM	Semantics, plus navigation noise	Strip <code>nav</code> , <code>footer</code> , <code>script</code> , then chunk on <code>h1</code> to <code>h3</code>
CSV	Schema	Column to value binding	Serialize each row with column names, add a table summary

Table 2.1: Loader selection by format, and the specific signal that naive extraction discards.

2.3 Cleaning and Normalization Without Destroying Signal

Normalization sits between parsing and chunking. Done well, it removes noise. Done carelessly, it removes meaning.

! Aggressive Regex Cleaning Destroys Retrieval

A common one liner strips every character outside `[A-Za-z0-9 .,!?-]`. That expression deletes currency symbols, percent signs, mathematical operators, code punctuation, and every non-Latin script. A finance corpus loses the difference between 15 and 15 percent. A codebase loses every bracket. Restrict cleaning to whitespace collapse, control character removal, Unicode NFKC normalization, and ligature repair, then stop.

The reference implementation in `chapter_02/loaders.py` originally applied exactly that destructive filter. The corrected version below preserves semantic characters while still removing genuine noise.

chapter_02/loaders.py, safe normalization

```

1 import re
2 import unicodedata
3
4 # Control characters excluding tab, newline, and carriage return.
5 CONTROL_CHARS = re.compile(r[\x00-\x08\x0b\x0c\x0e-\x1f\x7f])
6 # Ligature codepoints that PDF extraction commonly emits as single glyphs.
7 LIGATURES = {\uFB00: ff, \uFB01: fi, \uFB02: fl,
8              \uFB03: ffi, \uFB04: ffl}
9 # Hyphenation across a line break, for example retrie-\nval.
10 SOFT_HYPHEN_BREAK = re.compile(r(\w)-\s*\n\s*(\w))
11
12
13 def clean_text(text: str, preserve_layout: bool = True) -> str:
14     Normalize text without deleting semantically meaningful characters.
15
16     preserve_layout keeps paragraph breaks, which structure aware chunkers
17     depend on. Collapsing every newline into a space is the single most
18     common way teams destroy their own chunk boundaries.
19
20     for ligature, replacement in LIGATURES.items():
21         text = text.replace(ligature, replacement)
22
23     # Repair words split across a line break before collapsing whitespace.
24     text = SOFT_HYPHEN_BREAK.sub(r\1\2, text)
25
26     # NFKC folds compatibility forms, for example full width Latin.
27     text = unicodedata.normalize(NFKC, text)
28     text = CONTROL_CHARS.sub('', text)
29
30     if preserve_layout:
31         # Collapse runs of spaces and tabs, keep at most one blank line.
32         text = re.sub(r[ \t]+, , text)
33         text = re.sub(r\n{3,}, \n\n, text)
34     else:
35         text = re.sub(r\s+, , text)
36
37     return text.strip()

```

2.4 Metadata Extraction

Metadata is the filter that embeddings cannot express. A vector encodes what a chunk means. It does not encode when the chunk was written, which tenant owns it, which version supersedes it, or which clearance level it requires. Extract metadata at ingestion, because reconstructing it later requires reprocessing the corpus.

Metadata field	Source	Query time use
source_uri	Loader	Citation, deduplication, deletion
tenant_id	Ingestion request	Hard isolation filter, non-negotiable
acl_groups	Source system permissions	Per user retrieval filtering
doc_version	Source system or content hash	Suppress superseded chunks
effective_date	Document body or file meta-data	Recency boost, temporal filtering
section_path	Heading hierarchy	Citation precision, parent retrieval
page_number	Parser	Deep linking into the source PDF
content_hash	Computed	Idempotent upsert, duplicate suppression
language	Detector	Route to the right embedding model

Table 2.2: The metadata fields worth extracting at ingestion, with the query time capability each one unlocks.

Understanding these results and requirements provides clear operational context for the system design.

✓ Carry the Section Path Into the Chunk Text

Prefix each chunk with its heading breadcrumb, for example `Security Policy > Access Control > Contractor Accounts`. The breadcrumb costs 10 to 20 tokens, and it gives the embedding model the topical anchor that a mid document chunk otherwise lacks. Teams report 8 to 15 point recall gains from this change alone, because it repairs the most common ingestion defect at almost no cost.

2.5 Chunking Strategies

Chunking answers one question: what is the smallest unit of text that independently answers a question? Four strategies dominate production systems.

2.5.1 Fixed Size Chunking

Fixed size chunking slices the character stream every N characters with an overlap of M . It is deterministic, trivially parallel, and completely blind to meaning. It splits sentences, tables,

and code blocks without hesitation. Nevertheless, it remains the right choice for homogeneous, unstructured text such as chat transcripts, and it provides the baseline every other strategy must beat.

2.5.2 Recursive Character Chunking

Recursive chunking attempts a hierarchy of separators in order, typically paragraph break, then line break, then sentence boundary, then word boundary, then character. It splits on the coarsest separator that produces chunks under the size limit. Consequently, it respects paragraph structure most of the time while still guaranteeing a maximum size. This strategy is the correct default for prose.

2.5.3 Semantic Chunking

Semantic chunking embeds each sentence, measures the cosine distance between consecutive sentences, and inserts a boundary wherever the distance exceeds a percentile threshold. It detects topic shifts that no separator character marks. However, it costs one embedding call per sentence at ingestion, which is 20 to 60 times the cost of recursive chunking, and it produces highly variable chunk sizes that complicate token budgeting.

2.5.4 Parent Child Chunking

Parent child chunking indexes small children for precise matching, then returns the enclosing parent for generation. The child is 200 tokens, which makes its embedding sharp. The parent is 2,000 tokens, which gives the generator surrounding context. This strategy resolves the fundamental tension between retrieval precision and generation context, and it is the highest leverage upgrade available to most teams.

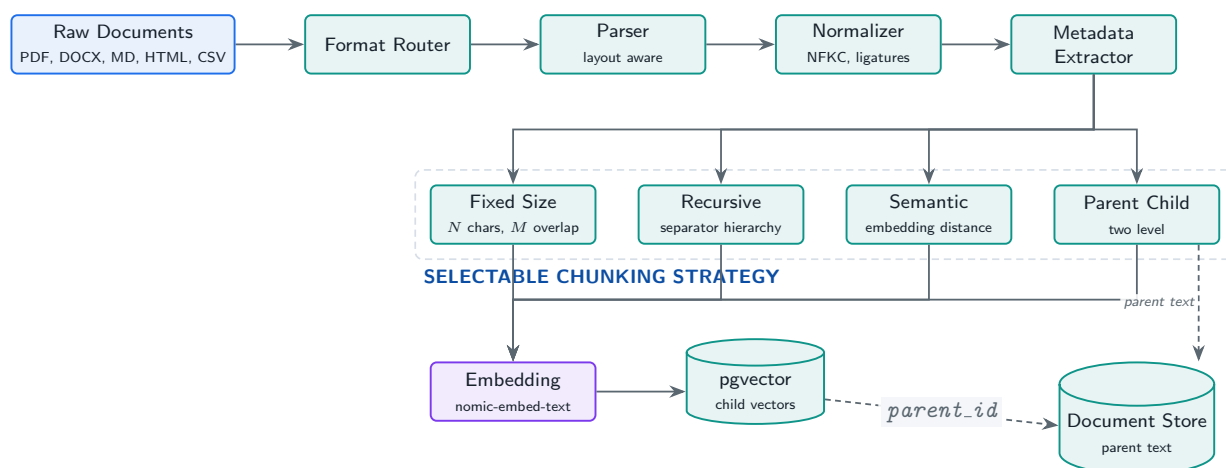


Figure 2.1: The ingestion pipeline with four chunking strategies branching from a shared parse and normalize stage. Parent child chunking writes to two stores, since children carry the vectors and parents carry the text.

Strategy	Strengths	Weaknesses	Ingest cost	Use when
Fixed size	Deterministic, fastest, uniform token budget	Splits sentences, tables, and code	1x	Homogeneous unstructured text, baseline
Recursive	Respects paragraphs, bounded size, cheap	Still blind to topic shifts	1.1x	Default for prose corpora
Semantic	Detects topic boundaries, coherent chunks	Costly, variable sizes, threshold sensitive	20 to 60x	Dense technical text, research corpora
Parent child	Precise matching plus rich context	Two stores, more complex deletion	1.3x	Long documents, most production systems
Structure aware	Uses real headings, best citation	Requires structured source formats	1.2x	Markdown, DOCX, HTML, and wikis

Table 2.3: Chunking strategy comparison. Ingest cost is relative to fixed size chunking on the same corpus.

2.6 Chunk Size and Overlap Trade-offs

Chunk size controls a precision and context trade-off that you should reason about in tokens rather than characters, since the generator’s budget is denominated in tokens.

Small Chunks Against Large Chunks

Small chunks of 150 to 300 tokens produce sharp embeddings, because a short passage has a single topic and its vector points clearly at that topic. They also fragment answers, so a single fact spread across three sentences may require three chunks. Large chunks of 1,000 to 2,000 tokens carry complete context, and their embeddings blur, because averaging many topics into one vector moves it toward the corpus centroid where it matches everything weakly. The empirical sweet spot for prose sits between 300 and 600 tokens, and parent child chunking lets you avoid the choice entirely.

Overlap exists to prevent an answer from falling exactly on a boundary. Ten to fifteen percent of chunk size is the standard range. Higher overlap raises recall marginally, and it inflates the index proportionally: 50 percent overlap doubles storage, doubles embedding cost, and fills the top K with near duplicate chunks that crowd out genuinely different evidence.

! Overlap Creates Duplicate Retrieval

With 50 percent overlap, adjacent chunks share half their content, so a query matching that shared region retrieves both. Your top 5 then contains three distinct passages rather than five. Deduplicate after retrieval by comparing chunk offsets from the same source, and merge chunks that overlap by more than 40 percent before building the context.

2.7 Building the Chunking Comparison Lab

The lab measures each strategy on the same document and reports chunk counts, size distributions, and boundary quality. The corrected implementation below replaces the placeholder semantic chunker in the repository with a real embedding based one.

Lab Code for This Chapter

Run the reference implementation from `chapter_02/chunkers.py` in the companion repository at github.com/lamhosiagian/rag-graphrag-lab. Run `streamlit run chapter_02/app.py`, upload a document, and compare all four strategies side by side.

chapter_02/chunkers.py, embedding based semantic chunking

```

1 import re
2 import numpy as np
3 from typing import List
4
5
6 SENTENCE_BOUNDARY = re.compile(r(?<=[.!?])\s+(?=[A-Z]))
7
8
9 def semantic_chunking(
10     text: str,
11     embeddings_model,
12     breakpoint_percentile: int = 85,
13     min_chunk_chars: int = 200,
14     max_chunk_chars: int = 1500,
15 ) -> List[str]:
16     Split on semantic discontinuity rather than on character count.
17
18     The algorithm embeds every sentence, measures cosine distance between
19     consecutive sentences, and cuts wherever the distance exceeds the given
20     percentile. A high percentile produces few large chunks, and a low
21     percentile produces many small ones, so treat it as a tuned parameter.
22
23     sentences = [s.strip() for s in SENTENCE_BOUNDARY.split(text) if s.strip()]
24     if len(sentences) < 3:
25         return [text] if text.strip() else []
26
27     # One batched call, not one call per sentence.
28     vectors = np.asarray(embeddings_model.embed_documents(sentences))
29     # L2 normalize so the dot product equals cosine similarity.
30     vectors /= np.linalg.norm(vectors, axis=1, keepdims=True) + 1e-10
31
32     similarities = np.sum(vectors[:-1] * vectors[1:], axis=1)
33     distances = 1.0 - similarities
34     threshold = float(np.percentile(distances, breakpoint_percentile))
35
36     chunks: List[str] = []
37     current: List[str] = [sentences[0]]
38     current_len = len(sentences[0])
39
40     for index, sentence in enumerate(sentences[1:]):
41         crosses_topic = distances[index] > threshold
42         too_long = current_len + len(sentence) > max_chunk_chars
43         long_enough = current_len >= min_chunk_chars

```

```

44
45     if (crosses_topic and long_enough) or too_long:
46         chunks.append( .join(current))
47         current = [sentence]
48         current_len = len(sentence)
49     else:
50         current.append(sentence)
51         current_len += len(sentence) + 1
52
53     if current:
54         chunks.append( .join(current))
55     return chunks

```

chapter_02/chunkers.py, parent child chunking with stable identifiers

```

1  import hashlib
2  from dataclasses import dataclass, field
3  from typing import Dict, List
4
5  from langchain_text_splitters import RecursiveCharacterTextSplitter
6
7
8  @dataclass
9  class ChildChunk:
10     A small, precisely embeddable unit that points back at its parent.
11     content: str
12     parent_id: str
13     child_id: str
14     order: int
15     metadata: Dict[str, str] = field(default_factory=dict)
16
17
18     def _stable_id(source_uri: str, text: str, position: int) -> str:
19         Content addressed identifier, which makes reingestion idempotent.
20         digest = hashlib.sha256(
21             f'{source_uri}|{position}|{text}.encode(utf-8)
22         ).hexdigest()
23         return digest[:24]
24
25
26     def parent_child_chunking(
27         text: str,
28         source_uri: str,
29         section_path: str = ,
30         parent_chars: int = 2000,
31         child_chars: int = 400,
32         child_overlap: int = 60,
33     ) -> tuple[Dict[str, str], List[ChildChunk]]:
34         Produce a parent lookup table plus the child chunks that get embedded.
35
36         Only children are embedded and searched. At query time the retriever
37         resolves each hit to its parent, deduplicates parents, and passes the
38         parent text to the generator. Precision comes from the child, and
39         context comes from the parent.
40
41         parent_splitter = RecursiveCharacterTextSplitter(
42             chunk_size=parent_chars,

```

```

43     chunk_overlap=0,
44     separators=[\n\n, \n\n, \n, . , ],
45 )
46 child_splitter = RecursiveCharacterTextSplitter(
47     chunk_size=child_chars,
48     chunk_overlap=child_overlap,
49     separators=[\n\n, \n, . , ],
50 )
51
52 parents: Dict[str, str] = {}
53 children: List[ChildChunk] = []
54
55 for parent_position, parent_text in enumerate(parent_splitter.split_text(text)):
56     parent_id = _stable_id(source_uri, parent_text, parent_position)
57     parents[parent_id] = parent_text
58
59     for child_position, child_text in enumerate(child_splitter.split_text(parent_text)):
60         # The breadcrumb anchors a mid document chunk to its topic.
61         anchored = f{section_path}\n{child_text} if section_path else child_text
62         children.append(
63             ChildChunk(
64                 content=anchored,
65                 parent_id=parent_id,
66                 child_id=_stable_id(source_uri, child_text, child_position),
67                 order=child_position,
68                 metadata={source_uri: source_uri, section_path: section_path},
69             )
70         )
71
72 return parents, children

```

2.8 Testing Ingestion Through the User Interface

The comparison application makes chunk boundaries visible, which is the only reliable way to catch a broken chunker. Follow this sequence.

1. Launch with `streamlit run chapter_02/app.py` and open `http://localhost:8501`.
2. Upload `test_data/sample.md`, set chunk size to 500 and overlap to 50, then press **Process**.
3. Open the **Chunk Inspector** tab and read the first three chunks of each strategy. Confirm that fixed size chunks start mid sentence while recursive chunks start at a paragraph boundary. That contrast is the entire argument for recursive splitting.
4. Open the **Statistics** tab and compare chunk count and average size. Semantic chunking should produce fewer, longer, and more variable chunks.
5. Re-run with overlap set to 250, half the chunk size. Observe the chunk count roughly double and inspect two adjacent chunks to see the duplicated text. That reproduces the duplicate retrieval pitfall.
6. Upload a PDF containing a table. Inspect whether any chunk preserves the header row alongside the data rows. Failure here is the ingestion ceiling made visible.

The diagram above illustrates the complete architectural pipeline and state interactions.

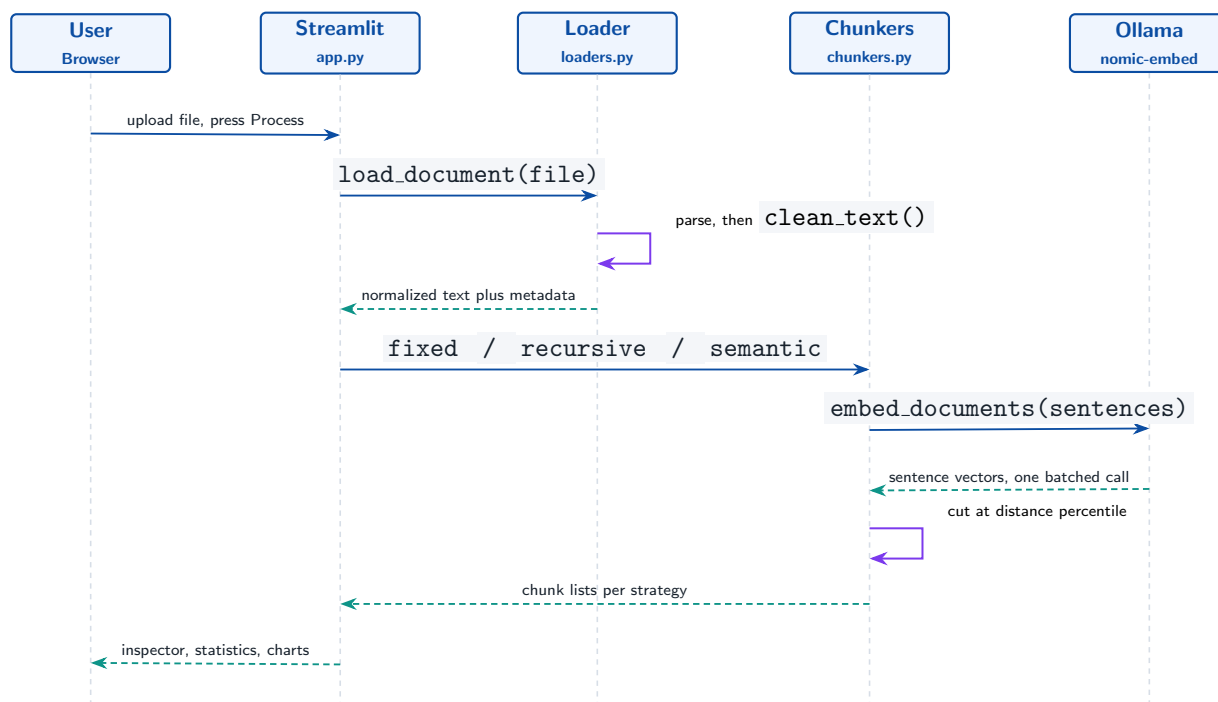


Figure 2.2: Ingestion sequence for a single uploaded document. Note that the embedding call is batched once per strategy rather than once per chunk, which is the difference between a 400 millisecond and a 12 second ingest.

Legal Contract Assistant, Table Loss Incident

A legal technology team indexed 12,000 commercial contracts with `pypdf` and recursive chunking. Questions about clause language answered well. Questions about payment terms failed constantly, and the team blamed the generator. Inspection of the chunks revealed the cause: payment terms lived in tables, and the stream extractor emitted table cells as a flat sequence of numbers with no column headers. A chunk read 30 60 90 2.5% 1.0% 0.5% with no indication of what any figure meant. Switching to a layout aware parser that rendered tables as Markdown, then repeating the header row into every table chunk, raised accuracy on numeric questions from 31 percent to 88 percent. No retriever or model change contributed to that gain.

2.9 Chapter Summary

Ingestion sets the ceiling on everything downstream. Choose a loader that preserves the structure your questions depend on, normalize conservatively so that currency symbols and code punctuation survive, and extract metadata at ingestion because you cannot recover it later. Chunk with recursive splitting as the default, upgrade to parent child chunking when documents are long, and reserve semantic chunking for corpora where topic drift within a document is genuine. Above all, measure the ingestion ceiling before you tune a retriever, since a retriever cannot find what the chunker destroyed.

2.10 Interview Questions and Answers

Q2.1 Design the ingestion pipeline for a 5 million document corpus with mixed formats

System Design

Q. Documents arrive continuously. Formats include scanned PDFs, native PDFs, DOCX, Confluence HTML, and CSV exports. Design the pipeline end to end.

Answer. Model the pipeline as a durable, idempotent, horizontally scalable job graph rather than a script.

Ingress publishes a change event per document containing the source URI, tenant, operation, and a content hash. Events land on a partitioned queue keyed by source URI, which guarantees that concurrent updates to the same document process in order and never interleave chunks.

Stage one classifies the document. Detect the MIME type, then for PDFs probe whether a text layer exists by extracting a sample page and measuring character density. Route text bearing PDFs to a layout aware parser and image only PDFs to an OCR path. Route DOCX and HTML to tree walkers. Route CSV to a row serializer. Emit the parse decision into the metadata so you can audit it later.

Stage two parses and normalizes. Preserve heading hierarchy as a section path, preserve tables as Markdown, and apply conservative normalization. Attach the metadata set from Table 2.2, especially tenant, ACL groups, version, and effective date.

Stage three chunks. Use structure aware splitting where a tree exists, and recursive splitting elsewhere. Produce parent and child units, and compute a content hash per child.

Stage four embeds and writes. Batch children into groups of 64, call the embedding service, and upsert on the content hash. Write parents to the document store in the same transaction boundary, or use an outbox pattern if the stores differ.

Handle deletion explicitly, because it is the stage most teams forget. A delete event must remove every chunk carrying that source URI, and it must run with higher priority than inserts, since a stale chunk from a deleted document creates compliance exposure.

Size the system by the slowest stage. OCR runs at roughly 2 pages per second per worker, and embedding runs at roughly 300 chunks per second per replica, so OCR usually dominates. Autoscale each stage independently on queue depth rather than on CPU, because queue depth is the signal that actually predicts backlog.

Finally, add a quality gate. After ingesting a document, run a sample of its chunks through a validator that checks for empty chunks, chunks that are pure whitespace or numbers, chunks exceeding the token limit, and chunks with a suspicious character distribution. Quarantine failures rather than indexing them.

Q2.2 Explain how you would select chunk size empirically

Technical Depth

Q. Do not give me a number. Give me the method that produces the number.

Answer. Treat chunk size as a hyperparameter tuned against a task specific golden set, because the optimum depends on the corpus and the question distribution rather than on any universal constant.

Build the golden set first. Sample 200 real questions from logs or from domain experts. For each question, have a human mark the exact span in the source corpus that answers it. Record the span length distribution, because it bounds the useful chunk size: if 90 percent of answer spans are under 250 tokens, chunks of 2,000 tokens carry mostly irrelevant text.

Run a sweep. For chunk sizes of 128, 256, 384, 512, 768, 1024, and 1536 tokens, with overlap fixed at 12 percent, reindex the corpus and measure four quantities. Measure the ingestion ceiling, meaning the fraction of answer spans fully contained in a single chunk. Measure recall at 10 and precision at 5 with the same retriever. Measure faithfulness of the generated answer. Measure index size and ingest wall clock.

Read the curves rather than the maximum. The ingestion ceiling rises monotonically with chunk size, because larger chunks contain more spans. Precision falls monotonically. Faithfulness usually peaks in the middle and falls at both extremes, since tiny chunks fragment evidence and huge chunks dilute attention. The best operating point sits where faithfulness peaks, not where recall peaks.

Segment the analysis. Compute the same curves separately for each document type. Contracts, chat transcripts, and API documentation almost always want different sizes, so a single global size is a compromise you should make consciously rather than by default.

Then remove the trade-off where possible. Parent child chunking decouples the embedded unit from the generated context, so you tune the child for precision and the parent for coverage, and the single number question disappears.

Finally, guard against drift. Store the chunking configuration alongside the index, re-run the sweep when the corpus composition changes materially, and treat a chunk size change as an index migration rather than a configuration edit.

Q2.3 Retrieval works for most documents and fails completely for a subset *Production Debugging*

Q. *Recall is 0.86 overall. For one document class it is 0.09. Where do you look?*

Answer. A recall cliff isolated to one document class almost always originates in ingestion rather than retrieval, so start by reading the chunks rather than the rankings.

Pull every chunk belonging to a failing document and print it verbatim. Six patterns explain nearly all such cliffs. First, empty or near empty chunks, which indicate a parser that returned nothing, common with scanned PDFs lacking a text layer. Second, chunks that are pure symbols or numbers, which indicate lost table structure. Third, chunks in a different language than the embedding model expects, which produces vectors near the corpus centroid. Fourth, chunks containing an entire document collapsed into one blob, which indicates the chunker never found a separator because normalization stripped every newline. Fifth, chunks containing navigation boilerplate repeated on every page, which makes every chunk from that source look identical. Sixth, chunks missing entirely, which indicates a silently

swallowed exception in ingestion.

Confirm the mechanism quantitatively. Compute the mean pairwise cosine similarity among chunks from the failing class. A value above roughly 0.95 tells you the chunks are near duplicates, typically boilerplate. Compute each chunk's similarity to the corpus centroid. Values clustered near the centroid tell you the embeddings carry little discriminating signal, typically because chunks are too long or the text is degraded.

Fix at the correct layer. Parser failures need a different parser or an OCR fallback. Boilerplate needs a stripping rule before chunking. Blob chunks need normalization that preserves paragraph breaks. Language mismatch needs either a multilingual embedding model or a translation step at ingestion.

Prevent recurrence with an ingestion quality gate that runs these same checks automatically on every document and quarantines failures, plus a per document class recall dashboard so the next cliff surfaces in hours rather than in a user complaint.

Q2.4 A wiki assistant returns the same navigation text for every question *Case Study*

Q. *Every answer cites the same three chunks, which contain the site menu. Explain the mechanism and the fix.*

Answer. This is boilerplate dominance, and the mechanism is a direct consequence of how similarity search works over a corpus with repeated text.

Wiki HTML carries navigation, breadcrumbs, sidebars, and footers on every page. Naive extraction includes them, so thousands of chunks share the same 200 words. Those chunks sit tightly clustered in vector space near the corpus centroid. A query vector, being roughly equidistant from everything it does not match strongly, lands closest to the densest cluster. Consequently, boilerplate wins whenever the query has no strong topical match, which is exactly the hard queries you most needed to answer.

Diagnose it in one query. Group chunks by a normalized hash of their text and count duplicates. Any text appearing in more than a few percent of chunks is boilerplate by definition.

Fix it at ingestion in three layers. First, strip structurally: remove `nav`, `header`, `footer`, `aside`, and `script` nodes before extracting text, and prefer the main content container when the site marks one. Second, strip statistically: compute shingles across the corpus, identify text blocks appearing in more than a threshold fraction of documents, and remove them regardless of tag. This catches boilerplate that carries no semantic markup. Third, filter at write time: reject chunks whose content hash already exists more than n times.

Add a retrieval side safeguard as defence in depth. Apply maximal marginal relevance so the top K favours diversity, and add a minimum score floor so a weak boilerplate match cannot enter the context at all.

Verify the fix by re-measuring the mean pairwise similarity of the index. A healthy corpus shows a broad distribution. A boilerplate heavy corpus shows a sharp spike, and that spike should disappear after the fix.

Q2.5 When is semantic chunking worth 40 times the ingestion cost?*Trade-offs*

Q. *Defend or reject semantic chunking for a specific workload.*

Answer. Justify it on the marginal accuracy it buys over recursive chunking for the specific corpus, not on its conceptual appeal.

Semantic chunking pays off when three conditions hold simultaneously. First, documents contain genuine topic drift without structural markers, meaning long unstructured prose such as transcripts, research narratives, or legacy reports where headings are absent. Second, the corpus is stable, so you pay the ingestion cost once and amortize it across millions of queries. Third, answer spans are topically coherent, which is what the boundary detector actually optimizes for.

It fails to pay off when documents already carry structure, because heading based splitting achieves the same boundaries for free and with better citations. It also fails on short documents, where the whole document fits in one chunk anyway, and on high churn corpora, where reingestion cost recurs constantly.

Quantify before committing. Run both strategies on the same golden set and compare faithfulness and recall. In most enterprise corpora the gap is 2 to 5 points, which rarely justifies the cost. In unstructured transcript corpora the gap can exceed 15 points, which clearly does.

Consider the cheaper middle path first. Recursive chunking with a heading aware separator list, plus a section path breadcrumb prefixed to each chunk, captures most of the benefit at 1.1 times the baseline cost. Parent child chunking captures a different and often larger share of the benefit at 1.3 times. Exhaust both before paying 40 times.

If you do adopt it, control the variance it introduces. Enforce minimum and maximum chunk sizes so token budgeting stays predictable, cache sentence embeddings keyed on the sentence hash so reingestion of a lightly edited document is cheap, and treat the percentile threshold as a tuned parameter measured on the golden set rather than a default.

Q2.6 How do you handle document updates and deletions without reindexing everything?*Production*

Q. *A 5 million document corpus changes by 2 percent daily. Full reindexing takes 40 hours. Design incremental maintenance.*

Answer. Build the index around content addressed identity so that incremental maintenance becomes the normal path and full reindexing becomes an exceptional one.

Assign every chunk a deterministic identifier derived from the source URI, the chunk position, and a hash of the chunk text. Ingestion then upserts rather than inserts. When a document changes, reprocess only that document, compute the new chunk identifier set, upsert the new set, and delete identifiers present previously but absent now. Because most edits touch a small part of a document, the majority of chunk hashes remain unchanged and their embeddings are reused, which typically cuts embedding cost on an update by 80 to 95

percent.

Drive the process from change data capture rather than from polling. Subscribe to source system webhooks where available, and fall back to a modified time scan with a watermark elsewhere. Record the watermark transactionally so a crash resumes rather than restarts.

Treat deletion as a first class, higher priority operation. Route delete events to a separate queue with its own workers so a large insert backlog cannot delay a deletion. Perform a soft delete first by flagging chunks as inactive and excluding them from retrieval, then hard delete asynchronously. Soft deletion makes the operation instantaneous from the user's perspective and reversible if the event was erroneous.

Keep the ability to rebuild, since embedding model upgrades and chunking changes require it. Build into a shadow table, validate against the golden set, then cut over atomically by swapping an alias or a partition. Never mutate the live index in place during a migration, because a half migrated index mixes vector spaces and fails silently, exactly as described in Chapter 1.

Finally, reconcile continuously. Run a nightly job that compares the set of source document identifiers against the distinct source identifiers in the index, and alert on drift in either direction. Drift detection catches the events you dropped, and every event driven system eventually drops events.

Q2.7 Why does adding the section heading to each chunk improve retrieval so much?

Technical Depth

Q. *Explain the mechanism, not just the empirical result.*

Answer. The mechanism is topical anchoring in embedding space, and it follows from how sentence embedding models are trained.

An embedding model maps text to a point whose position encodes topic, entities, and intent. A mid document chunk frequently lacks explicit topic markers, because the document established the topic in a heading several paragraphs earlier and subsequent text relies on pronouns and implicit reference. A chunk reading “Accounts must be disabled within 24 hours of the trigger event” contains no indication that the subject is contractor offboarding. Its embedding therefore lands in a generic region about accounts and timing, far from a query about contractor access revocation.

Prefixing the breadcrumb “Security Policy › Access Control › Contractor Accounts” injects exactly the missing signal. The chunk's vector shifts toward the region occupied by queries about contractor access, and cosine similarity with such queries rises sharply. Importantly, this is not a keyword trick, since the effect persists for paraphrased queries that share no tokens with the heading.

Two secondary benefits follow. Citation precision improves, because the answer can name the exact section rather than the document. Reranking improves, because a cross encoder sees the topical context and scores the pair more accurately.

Watch the two failure modes. If breadcrumbs are long, they consume a meaningful fraction of a small chunk and dilute the body text, so cap them at roughly 15 tokens and keep only the deepest two or three levels. If every chunk in a large document shares one breadcrumb, it adds a constant vector offset that slightly compresses the distinction among those chunks, which is harmless at typical corpus scale but worth measuring.

Validate the change the same way you validate any retrieval change: reindex a copy, run the golden set, and compare recall at 10 and faithfulness. In most corpora this single change delivers a larger gain per engineering hour than any retriever modification available.

Embeddings and Vector Database Architecture

Subscribe → aiengineeringinsider.substack.com/subscribe

What This Chapter Delivers

This chapter turns retrieval from a black box into an engineering discipline. You will learn what an embedding actually represents, how dense and sparse representations differ in the failures they produce, why cosine similarity and the dot product are not interchangeable, and how approximate nearest neighbour indexes trade recall for speed. You will then design a pgvector schema that supports metadata filtering, tenant isolation, and index maintenance under load, and you will build a semantic search engine that exposes every one of those levers.

3.1 What an Embedding Represents

An embedding model maps a text span to a fixed length vector such that semantically similar spans land near each other. Training achieves this through a contrastive objective: the model sees pairs known to be related, pulls their vectors together, pushes unrelated pairs apart, and repeats across hundreds of millions of pairs. Consequently, the geometry of the resulting space encodes the notion of similarity present in the training pairs, and nothing more.

★ Similarity Is Defined by Training Data, Not by Intuition

An embedding model trained on question and answer pairs places a question near its answer. A model trained on paraphrase pairs places a question near a rephrasing of the same question. These are different geometries. Therefore, a model that excels at symmetric similarity may perform poorly at asymmetric retrieval, where a short query must match a long passage. Always check whether a model was trained for retrieval, and whether it expects query and document prefixes.

3.1.1 Asymmetry Between Queries and Documents

Retrieval is asymmetric. A query is short, interrogative, and often incomplete. A document chunk is long, declarative, and self contained. Models built for retrieval handle this by training on exactly that asymmetry, and several expose instruction prefixes to signal which side of the pair a text belongs to.

The `nomic-embed-text` model used throughout this book supports task prefixes such as `search_query:` and `search_document:`. Applying them consistently is not optional decoration. Omitting them, or applying them inconsistently between indexing and querying, measurably degrades recall.

! Prefix Skew Is a Silent Recall Killer

If ingestion embeds chunks with a document prefix while the serving path embeds queries without any prefix, the two sides occupy slightly different regions of the space. Scores remain plausible, so nothing raises an alarm, and recall falls by 5 to 15 points. Store the exact prefix convention alongside the embedding model version in your index metadata, then assert it at startup.

3.2 Dense and Sparse Representations

Two families of representation dominate retrieval, and mature systems use both.

Property	Dense vectors	Sparse vectors
Dimensionality	384 to 3,072, all non-zero	Vocabulary sized, mostly zero
Produced by	A neural encoder	Term statistics such as BM25, or a learned sparse model
Captures	Meaning, paraphrase, cross-lingual relations	Exact terms, rare tokens, identifiers
Fails on	Exact identifiers, product codes, acronyms, negation	Paraphrase, synonymy, conceptual questions
Index	HNSW, IVF, or exhaustive scan	Inverted index
Interpretability	Low, since dimensions carry no names	High, since dimensions are terms
Cost to build	GPU or CPU inference per chunk	Tokenization and counting only

Table 3.1: Dense and sparse representations differ most in the queries they fail on, which is why combining them in Chapter 5 raises recall substantially.

Understanding these results and requirements provides clear operational context for the system design.

Support Portal, Error Code Retrieval

A hardware vendor indexed 90,000 support articles with dense retrieval only. Conceptual questions such as “why does the device restart under load” answered well. Queries containing an error code such as `ERR_0x8007005A` failed almost entirely, because the tokenizer split the code into meaningless subword fragments and the encoder had never seen that token during training. The vector therefore carried no information about the specific code. Adding BM25 alongside the dense retriever, fused with reciprocal rank fusion, raised exact-code recall from 0.11 to 0.94 while leaving conceptual recall unchanged.

3.3 Similarity Measures

Three measures appear in practice, and confusing them produces subtle ranking bugs.

3.3.1 Cosine Similarity

Cosine similarity measures the angle between two vectors and ignores their magnitudes.

$$\cos(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \|\mathbf{d}\|}$$

It is the correct default for text retrieval, because vector magnitude in most encoders correlates with text length rather than with relevance. Ignoring magnitude therefore prevents long chunks from dominating simply because they are long.

3.3.2 Dot Product

The dot product multiplies cosine similarity by both magnitudes.

$$\mathbf{q} \cdot \mathbf{d} = \|\mathbf{q}\| \|\mathbf{d}\| \cos(\mathbf{q}, \mathbf{d})$$

It is faster, since it skips two norm computations, and it is correct only when vectors are already L2 normalized, in which case it equals cosine similarity exactly. Some models are trained specifically so that magnitude encodes a quality or confidence signal, and for those models the dot product is the intended measure. Read the model card rather than assuming.

3.3.3 Euclidean Distance

Euclidean distance measures straight line separation. On L2 normalized vectors it is a monotone function of cosine similarity, so ranking is identical. On unnormalized vectors the rankings diverge. Prefer cosine unless you have a specific reason otherwise.

✓ Normalize Once at Write Time

Normalize every vector to unit length before storing it. Afterwards, cosine similarity, dot product, and Euclidean ranking all agree, index build becomes cheaper, and you eliminate an entire class of bugs where two code paths use different measures. In pgvector, store normalized vectors and use the `vector_cosine_ops` operator class so the index and the query agree.

3.4 Vector Indexing and Approximate Search

Exhaustive search computes similarity against every vector. It returns perfect results, and its cost scales linearly, so it becomes untenable somewhere between one hundred thousand and one million vectors depending on hardware. Approximate nearest neighbour indexes trade a small amount of recall for orders of magnitude in speed.

3.4.1 HNSW

Hierarchical Navigable Small World builds a layered proximity graph. The top layer contains few nodes with long range links, and each descending layer adds nodes and shorter links. A search enters at the top, greedily walks toward the query, then descends and repeats. The structure gives logarithmic-like search behaviour with high recall.

Three parameters govern the trade-off. The parameter `m` sets the number of bidirectional links per node, which controls graph connectivity and memory. The parameter `ef_construction` sets the size of the candidate list during build, which controls index quality and build time. The parameter `ef_search` sets the candidate list size at query time, which trades latency for recall and, importantly, is tunable per query without rebuilding.

3.4.2 IVFFlat

Inverted File with Flat compression clusters vectors into lists using k-means, then searches only the lists nearest the query. It builds far faster than HNSW and uses less memory. However, it requires representative data at build time, its recall degrades as the data distribution drifts away from the original centroids, and it needs periodic rebuilding.

Property	Exhaustive	IVFFlat	HNSW
Recall	1.00 by definition	0.85 to 0.97, probe dependent	0.95 to 0.99, <code>ef_search</code> dependent
Query latency at 1M	300 to 900 ms	5 to 20 ms	1 to 8 ms
Build time at 1M	None	Minutes	Tens of minutes
Memory over-head	None	Low	2 to 4 times the raw vectors
Incremental inserts	Free	Degrades until re-build	Handled natively
Tunable at query time	Not applicable	<code>probes</code>	<code>ef_search</code>
Best for	Under 100k vectors, exact ground truth	Large static corpora, memory constrained	Default for production RAG

Table 3.2: Index selection for pgvector. Latency figures assume 768 dimensional vectors on commodity server hardware and should be re-measured on your own workload.

3.5 Metadata Filtering and the Pre-filter Problem

Metadata filtering looks trivial and it is the most common source of silent recall loss in vector search.

3.5.1 Post-filtering Collapses Recall

The naive approach retrieves the top K by vector similarity, then discards results failing the filter. Suppose a tenant owns 0.5 percent of the corpus. Retrieving the top 100 globally and then filtering leaves roughly zero surviving rows, so the query returns nothing despite the tenant having perfectly relevant documents.

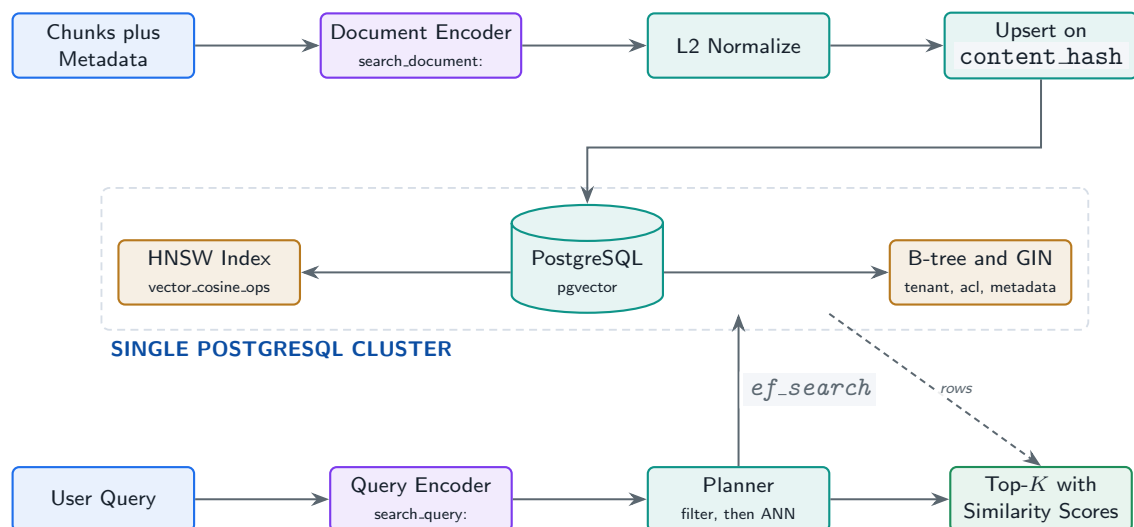


Figure 3.1: Vector database architecture for a multi-tenant RAG system. Filtering happens before the ANN traversal through a partial index, which avoids the post-filter recall collapse described below.

3.5.2 Pre-filtering Breaks the Graph

The opposite approach filters first, then searches within the surviving set. Semantically this is correct. Practically, an HNSW graph built over the full corpus cannot be traversed over an arbitrary subset, because the links needed to navigate may point at excluded nodes, so the walk gets trapped.

3.5.3 The Three Workable Strategies

- 1. Partitioned indexes.** Build a separate HNSW index per tenant or per high cardinality partition, using PostgreSQL partial indexes or table partitioning. Filtering then selects an index rather than filtering rows, so recall stays intact. This is the correct default for multi-tenant systems.
- 2. Filtered traversal.** Use an engine that evaluates the predicate during graph traversal, keeping the walk inside the allowed set. Recent pgvector versions support this through iterative index scans, and it works well for moderate selectivity.
- 3. Over-retrieval with a bounded loop.** Retrieve $K \times f$ candidates, filter, and increase f until you have enough survivors or hit a cap. It is simple and it degrades badly on highly selective filters, so treat it as a fallback rather than a design.

! Tenant Isolation Is Not a Filter, It Is a Boundary

Never rely on an application supplied `WHERE tenant_id = $1` alone. A single missing predicate in one code path leaks another customer's documents into an answer. Enforce isolation with PostgreSQL row level security so the database refuses cross-tenant reads even when the application forgets, and add partitioning so the physical layout matches the security boundary.